Testing ON DELETE CASCADE

Lecture Notes · Game DB Junior

In this lesson

- How to delete a whole table with **DROP TABLE**
- Where to write an **ON DELETE** option
- Testing **ON DELETE CASCADE** and when to use it

1. The plan

Our goal is to try the ON DELETE options and understand them with examples. Keep the CREATE TABLE and INSERT SQL from the last lecture somewhere safe; we use it several times.

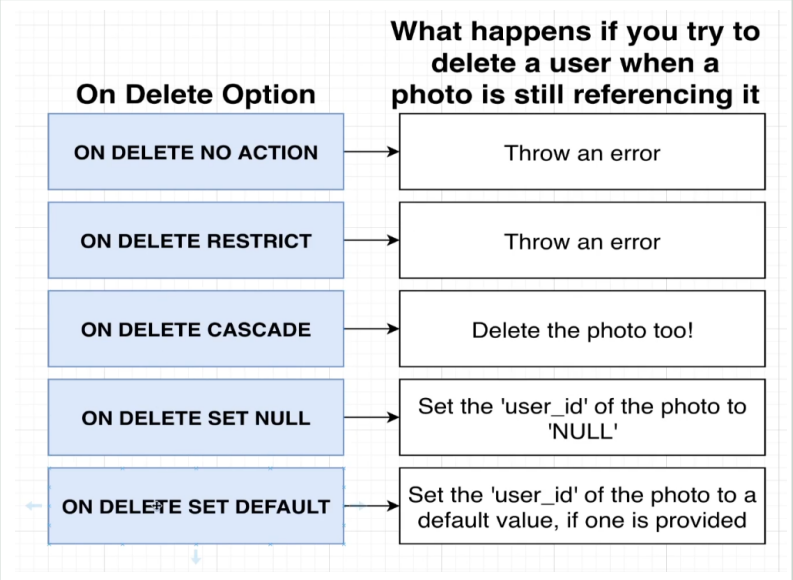


Figure 1 — The ON DELETE options

- First we **drop** the photos table. In SQL, **drop** means **delete**: the whole table and all its photos are deleted.
- Then we create the table again and insert data. Each time we create it, we set up one ON DELETE option.

2. DROP TABLE

```
DROP TABLE photos;
```

- After this, the photos table no longer exists.
- If we now run **SELECT * FROM photos;** we get an error: **relation "photos" does not exist**, because we just deleted the table.

3. Where to write ON DELETE

We write the phrase **right after the foreign key column** when we create the table. Here, after the user_id column:

```
CREATE TABLE photos (
    id SERIAL PRIMARY KEY,
    url VARCHAR(200),
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE
);
```

That's it. Then we run the CREATE TABLE and the INSERT statements together.

4. Testing ON DELETE CASCADE

Reminder: if we delete the user with id 1, all the photos tied to that user (photos 2, 3 and 4) should be deleted **automatically**.

```
DELETE FROM users
WHERE id = 1;
```

- The delete is successful.
- Behind the scenes, Postgres also deleted the related photos.
- With **SELECT * FROM photos;** we see that no photos of user 1 are left.

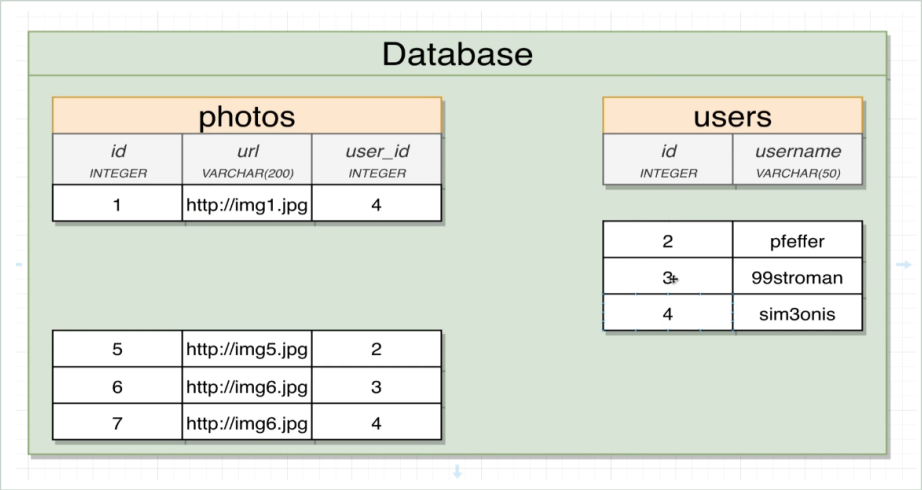


Figure 2 — After deleting user 1, photos 2, 3 and 4 are gone

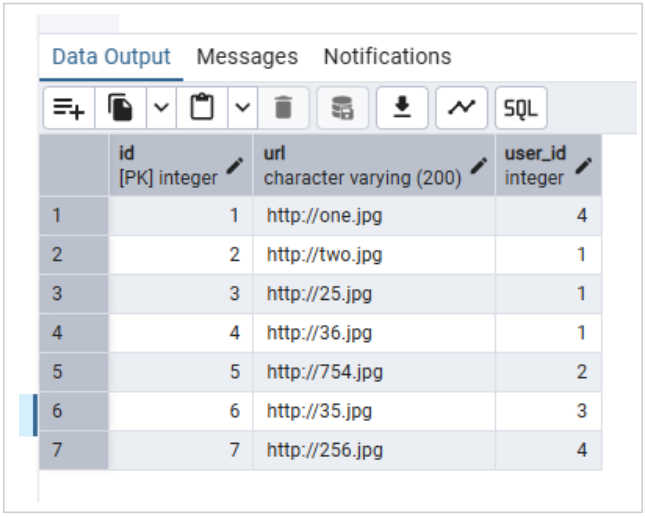
5. When to use CASCADE

You will use ON DELETE CASCADE **very often**: any time you delete a record that has many dependent records.

- **Discussion forum:** a post has many replies. If you delete a post (a main thread), you probably want to delete all its replies too.
- **Blog post:** a blog post has many comments. If you delete the post, there is no reason to keep its comments, because no one will read them.

6. Running the code (my practice)

Before: the photos table has 7 photos.



Data Output | Messages | Notifications

	id [PK] integer	url character varying (200)	user_id integer
1	1	http://one.jpg	4
2	2	http://two.jpg	1
3	3	http://25.jpg	1
4	4	http://36.jpg	1
5	5	http://754.jpg	2
6	6	http://35.jpg	3
7	7	http://256.jpg	4

Figure 3 — The photos table before DROP

Step 1: DROP TABLE photos. Now only cities and users are left.

```
-- Delete (drop) the whole photos table and all the data in it
-- After this, "SELECT * FROM photos" gives an error: the table does not exist
DROP TABLE photos;
```

Figure 4 — DROP TABLE photos

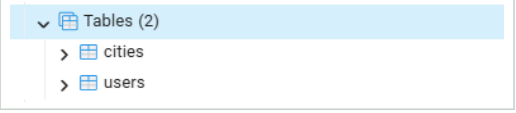


Figure 5 — Tables (2)

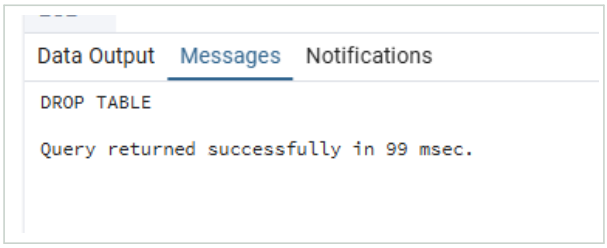


Figure 6 — DROP TABLE ran successfully

Step 2: SELECT * FROM photos gives an error, because the table does not exist.

```
-- Get all rows and all columns from the photos table
-- Right after DROP TABLE photos, this gives an error:
-- "relation photos does not exist" (the table was deleted)
SELECT * FROM photos;
```

Figure 7 — SELECT after DROP

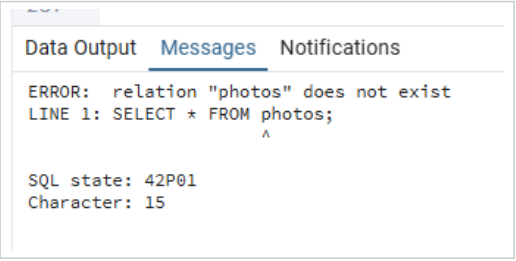


Figure 8 — ERROR: relation "photos" does not exist

Step 3: create the photos table again with ON DELETE CASCADE, and insert 7 photos.

```sql
-- Create the photos table again (after dropping it)
CREATE TABLE photos (
    id SERIAL PRIMARY KEY,
    url VARCHAR(200),
    -- ON DELETE CASCADE: when a user is deleted,
    -- all photos of that user are deleted too
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE
);

-- Add 7 photos to the new photos table
INSERT INTO photos (url, user_id)
VALUES
    ('http:/one.jpg', 4),    -- id 1, user 4
    ('http:/two.jpg', 1),    -- id 2, user 1
    ('http:/25.jpg', 1),     -- id 3, user 1
    ('http:/36.jpg', 1),     -- id 4, user 1 (user 1 has 3 photos)
    ('http:/754.jpg', 2),    -- id 5, user 2
    ('http:/35.jpg', 3),     -- id 6, user 3
    ('http:/256.jpg', 4);    -- id 7, user 4 (user 4 has 2 photos)
```

Figure 9 — CREATE TABLE with ON DELETE CASCADE, then INSERT

The Messages tab shows **INSERT 0 7**. Now there are 3 tables again.

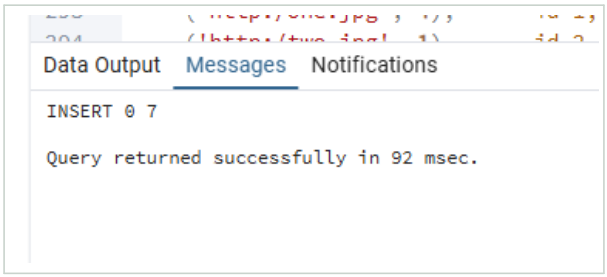


Figure 10 — INSERT 0 7

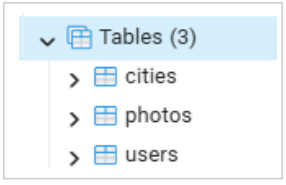


Figure 11 — Tables (3)

Step 4: delete the user with id 1. The Messages tab shows **DELETE 1**.

```sql
-- Delete the user with id 1 from the users table
-- Because of ON DELETE CASCADE, Postgres also deletes
-- all photos of this user (photos with id 2, 3 and 4)
DELETE FROM users
WHERE id = 1;    -- only the row where id is 1
```

Figure 12 — DELETE FROM users WHERE id = 1

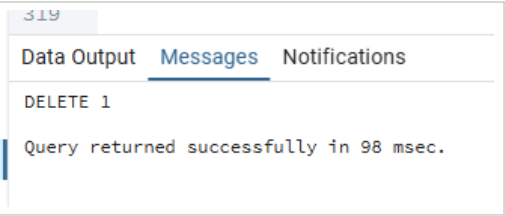


Figure 13 — DELETE 1

Step 5: SELECT * FROM users. User 1 is gone; users 2, 3, 4 are left.

```
SELECT * FROM users;
```

Data Output | Messages | Notifications

	id [PK] integer	username character varying (50)
1	2	pferrer
2	3	si93onis
3	4	99stroman

*Figure 14 — SELECT * FROM users*

Figure 15 — 3 users left

Step 6: SELECT * FROM photos. Photos 2, 3 and 4 (of user 1) were deleted too. Photos 1, 5, 6, 7 are left.

```
SELECT * FROM photos;
```

Data Output | Messages | Notifications

	id [PK] integer	url character varying (200)	user_id integer
1	1	http:/one.jpg	4
2	5	http:/754.jpg	2
3	6	http:/35.jpg	3
4	7	http:/256.jpg	4

*Figure 16 — SELECT * FROM photos*

Figure 17 — 4 photos left

7. Summary

Idea	Remember
DROP TABLE name	Deletes the whole table and all its data
Where ON DELETE goes	Right after the foreign key column, e.g. REFERENCES users(id) ON DELETE CASCADE
ON DELETE CASCADE	Deleting a user also deletes all their photos
Good use cases	Forum post and replies, blog post and comments

Next lesson: ON DELETE SET NULL.

Sources: Lecture transcript and lecture images (Figures 1–2); my own practice screenshots (Figures 3–17)